



MLOps : GPT-2 Fine-tuning Platform



Martin Kaiser

Tannaz Mostafid

Project Overview

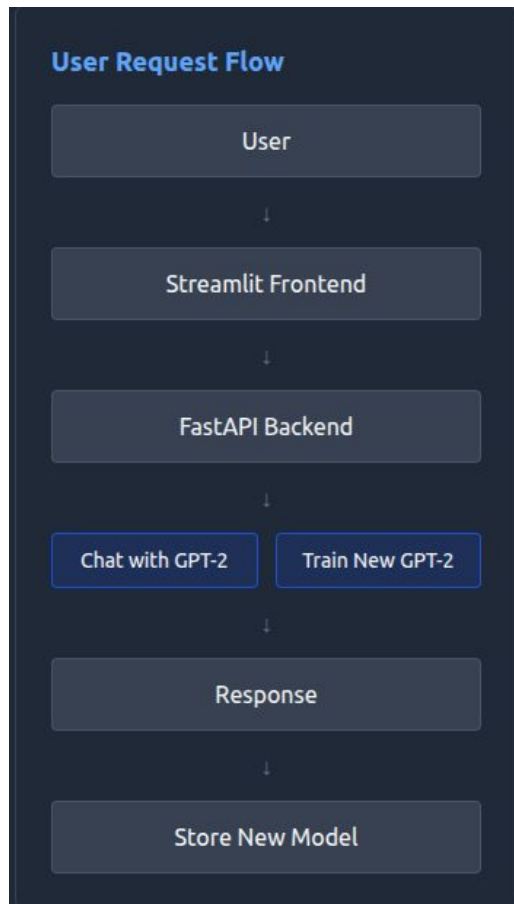
What We Built

- A fully containerized ML platform on Azure



Key Features:

- **Personalized Training:** Finetune your own GPT model with custom data (e.g., Shakespeare, Sci-fi, etc)
- **Cost Optimized:** GPU nodes auto-scale, turn off when idle for min costs
- **Kubernetes Resilient:** Automatic pod restart, self-healing, and high availability of app services
- **Self-Managed:** Kubeadm cluster with automated job transfer to GPU and self-healing orchestration
- **User-Friendly:** Streamlit app provides easy, intuitive functionality



Project Overview: How Our tools work together

Infrastructure Layer

Terraform – automates provisioning

Azure Cloud – VMs, VNet, storage

Orchestration Layer

Kubernetes – schedules and manages workloads

Docker – packages applications

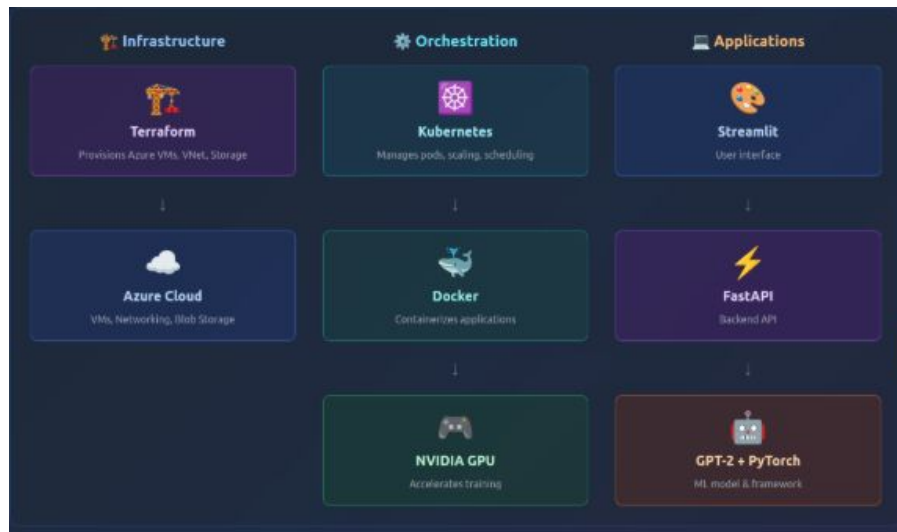
GPU Node – enabled with NVIDIA runtime

Application Layer

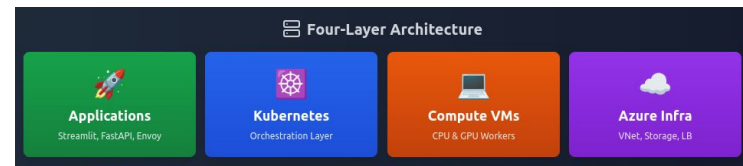
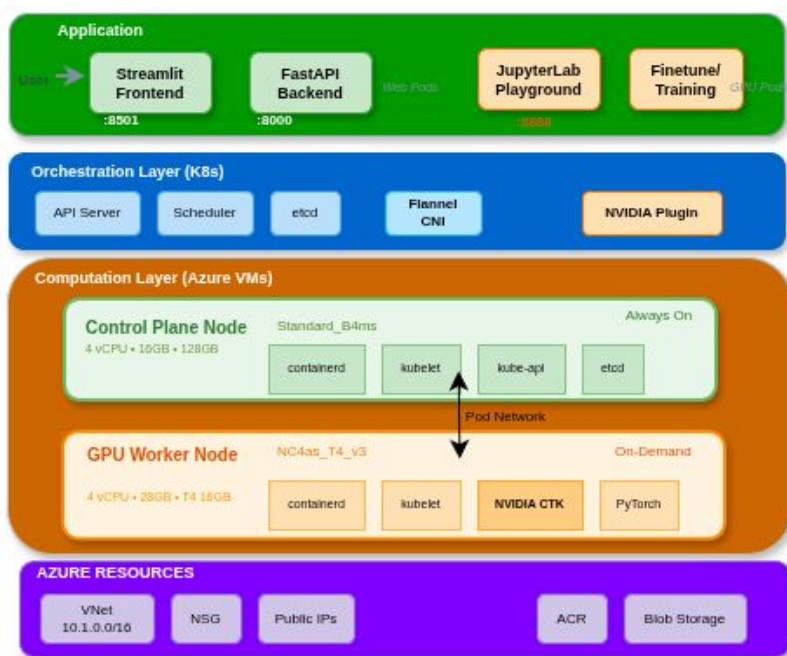
Streamlit – user interface

FastAPI – backend API

GPT-2 + PyTorch – model & inference



Architecture Overview



Layer 1: Applications

User-facing services including Streamlit Frontend, FastAPI Backend, and Envoy Gateway for intelligent routing

Layer 2: Kubernetes Orchestration

Manages workload scheduling, service discovery, API gateway, GPU plugins, and health monitoring

Layer 3: Compute VMs

Separate VM pools for CPU-intensive (Standard_B2ms) and GPU workloads (NC4as_T4_v3) with specialized configurations

Layer 4: Azure Infrastructure

Virtual Network, Subnet Control, Storage, Load Balancers, and networking components

Step 1: Setup Nodes, pods, and workloads

Setup

Two nodes working together: Control Plane runs our web app (Streamlit + FastAPI), GPU Worker runs all ML training jobs.

How It Works

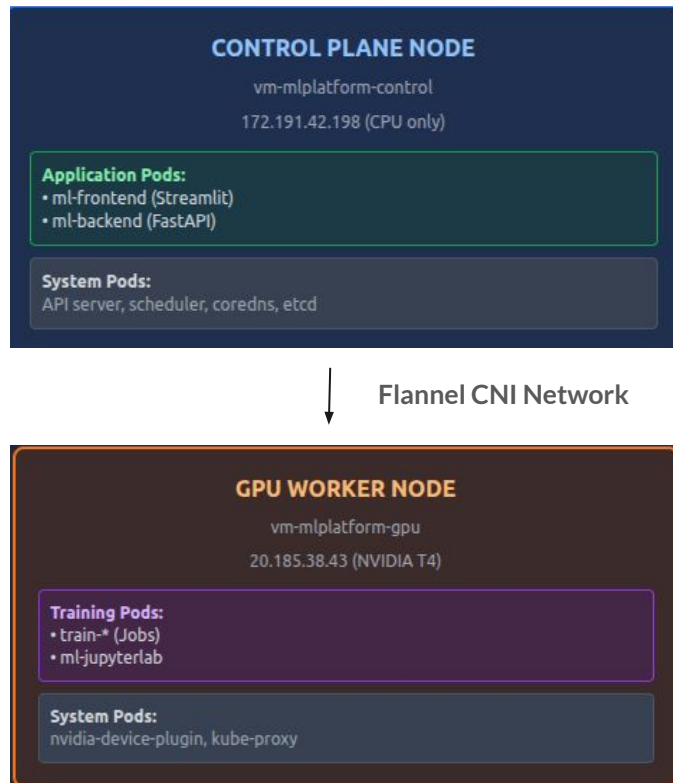
Chat: User → Frontend → Backend → Response

Training: User → Backend → GPU Job → Model Saved

Jobs automatically scheduled to GPU node using tolerations

Why This Design?

- CPU node handles lightweight UI/API tasks
- GPU node dedicated to expensive ML training
- **Cost optimized:** GPU only used when needed
- Industry standard production pattern



Step 2: GPU Enablement

- **NVIDIA Driver**

Installed on NC4as_T4_v3 VMs, Validate with nvidia-smi

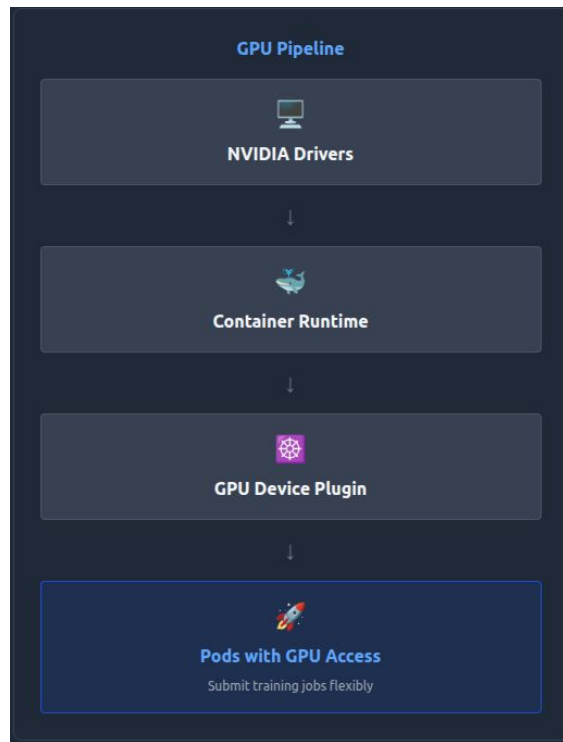
- **Container Toolkit**

Enabled GPU access from Docker container

- **GPU Operator**

Exposed GPUs as schedulable Kubernetes resources,

Enables flexible training job submission inside K8s cluster.



Step 3: Application Layer

- **Frontend & Backend**

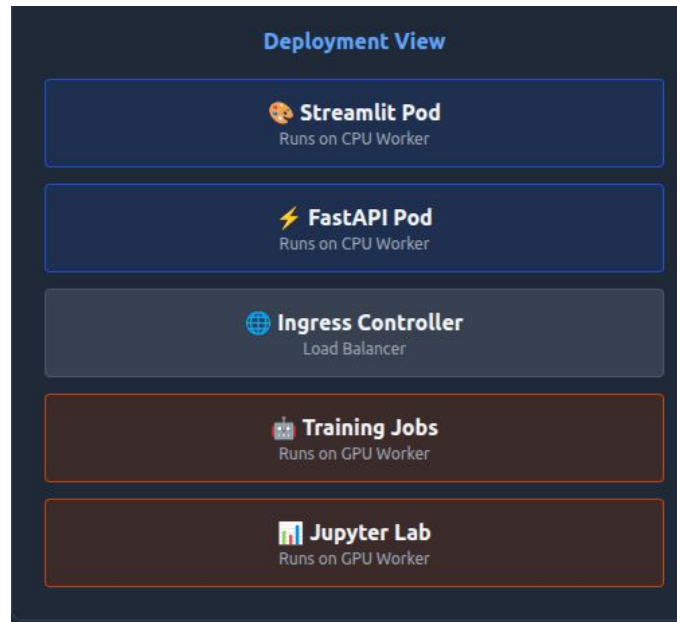
Deployed Streamlit UI and FastAPI services as Kubernetes pods

- **Ingress Controller**

Manages external access and routing to services

- **ML Workloads**

Jupyter Lab and training job on GPU nodes



Step 4: Terraform IaC

- **Azure Resources**

VNet, Subnets, NSGs, Load Balancer provisioned via codes

- **Network Security Groups**

Self-managed Kubernetes requires NSG rules to be allowed (API Server, Nodeports Flannel etcd,...)

- **VM Configurations**

Separate Configs for control plane,CPU and GPU workers

- **State Management**

Remote state in Azure Storage with locking

Infrastructure Stack

`terraform apply`

Provisions all Azure resources

 **Virtual Network**

10.0.0.0/16

 **NSG Rules**

API: 6443, NodePort: 30000-32767, Flannel: 8472, etcd: 2379-2380

 **VM Scale Sets**

Control + Workers

 **Load Balancer**

Traffic distribution

Challenges and Solutions

Version Management

Package/Driver Configuration

Kubeadm version constraints (± 1 minor), NVIDIA driver compatibility, CNI setup complexity

→ **Version matching and reconfiguration when needed**

Authentication Management

Certificate Invalid for Public IP

kubectrl fails with TLS error. Cert valid for 10.96.0.1, 10.1.1.10, not 172.191.42.198

→ **SSH to control plane OR regenerate certs with --apiserver-cert-extra-sans**

Pod ImagePullBackOff

→ **Check ACR secret authentication**

Node NotReady

→ **sudo systemctl status kubelet**

Storage Management

Blob Storage Sync Delay

Standard blob storage updates every 120 seconds. Training data and models need faster access.

→ **Force updates in FastAPI backend pod**

GPU Configuration

GPU not visible

→ **kubectrl get pods -n kube-system | grep nvidia**

nvidia-device-plugin stuck

→ **Install NVIDIA Container Toolkit on GPU node**

Networking

Failed to find plugin loopback

→ **Install CNI plugins v1.3.0, copy to /opt/cni/bin**

Demo Time: Let's see it in action!



Backup slides

K8s ML Platform - Layered Architecture

